



Introduction to NLP

AIAA 4051 Lecture 20

Instructor: Sihong Xie

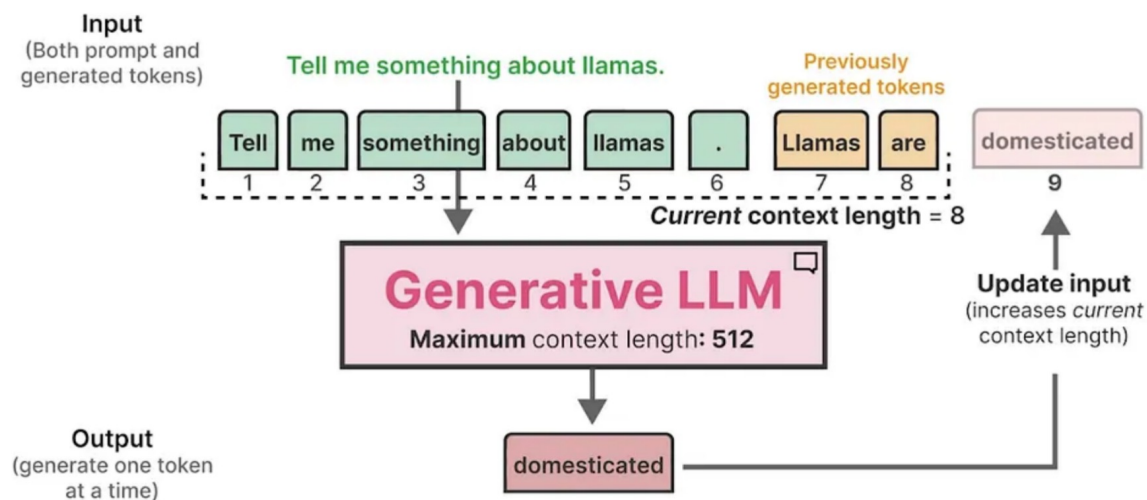
Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

Motivation of KV Cache

- In a naïve implementation, autoregressive generation would waste massive compute power recalculating the entire historical context (**K and V matrices**) just to predict one single new token.

- Input: "Tell me something about llamas. Llamas are"
- Step 1: Predict "**domesticated**"
- Context of Step 2: "Tell me something about llamas. Llamas are **domesticated**"



Quadratic Growth

To generate the i -th token, you recompute the Key and Value vectors for tokens 1 through $i-1$. These vectors have not changed since the last step! The time complexity is $O(n^2)$ when generating n tokens (ignoring the factor of the cost of computing the attention matrix).

Image source: <https://blog.gopenai.com/anatomy-of-llms-transformer-overview-encoder-decoder-autoregressive-flow-6a59bc839710>

Incremental Attention with Cache

- Recall the core attention formula for a sequence length L :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- During naive decoding, $Q, K, V \in R^{L \times D}$ are all recomputed every time when L increases by 1.
- With KV cache, at step t , we only compute vectors for the latest generated token and update the cache:

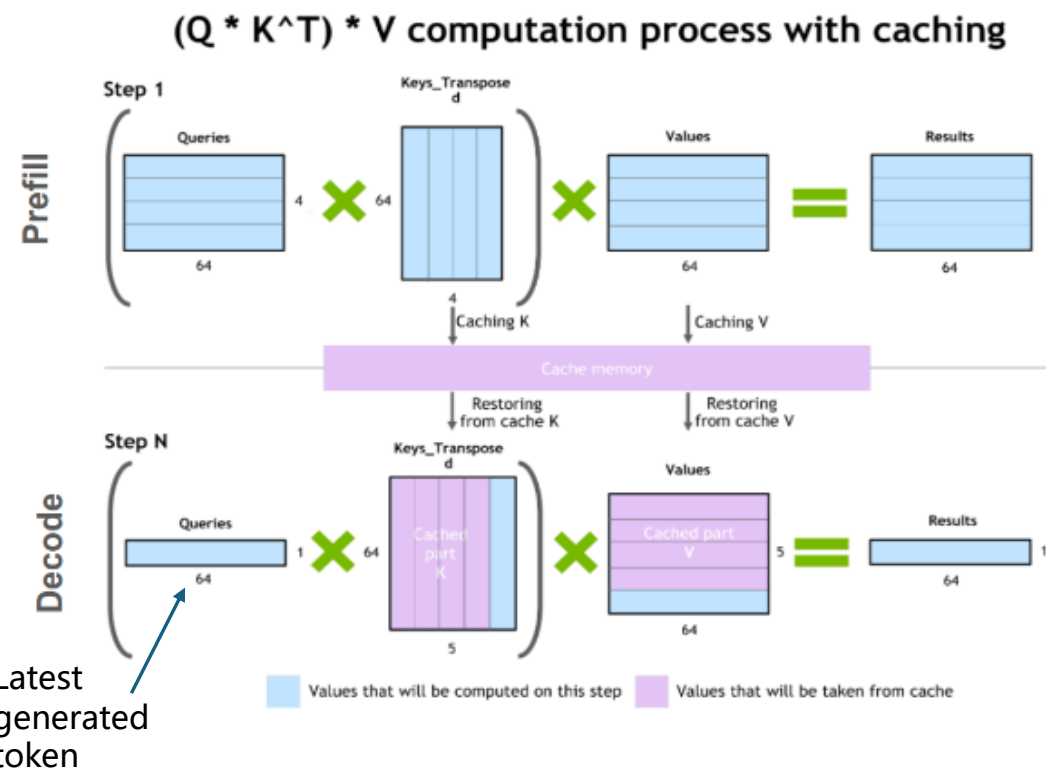
$$K_{\text{cached}} = [K_{\text{prev}}; k_{\text{new}}], \quad V_{\text{cached}} = [V_{\text{prev}}; v_{\text{new}}]$$

- The attention score for the query q_t (from the latest generated token):

$$\text{Score}_t = \text{softmax}\left(\frac{q_t K_{\text{cached}}^T}{\sqrt{d_k}}\right) V_{\text{cached}}$$

What is Key-Value (KV) Cache?

- Trading Memory for Time
- Instead of recomputing the Key (K) and Value (V) matrices from previous tokens from scratches, we store them in GPU memory.
- Compute once: $K_{1...t}, V_{1...t}$
- Cache: $K_{1...t}, V_{1...t}$
- Used for generating: $q_{t+1}, k_{t+1}, v_{t+1}$
- Result: $O(n)$ computation instead of $O(n^2)$.



A KV Cache saves results of intermediate steps while solving a problem. The next time you need to use those same intermediate values; you can simply look them up from the cache.

Image source: <https://ankitbko.github.io/blog/2025/08/prompt-engineering-kv-cache/>

The Two Stages of Token Generation

- Most LLMs generate text token by token.
- The first output token depends on all prompt tokens, but the second output token already depends on all prompt tokens plus the first output token, and so on.

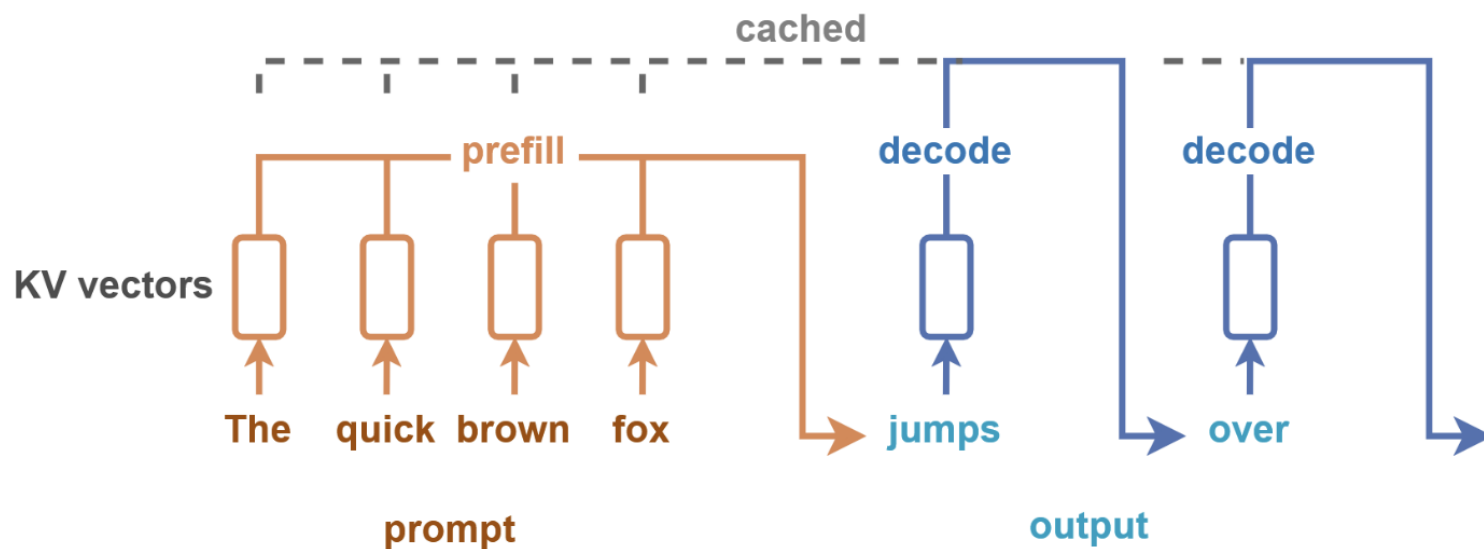


Image source: <https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests>

Phase 1: Prefill (processing the input)

- Encode the full prompt and build the initial cache.

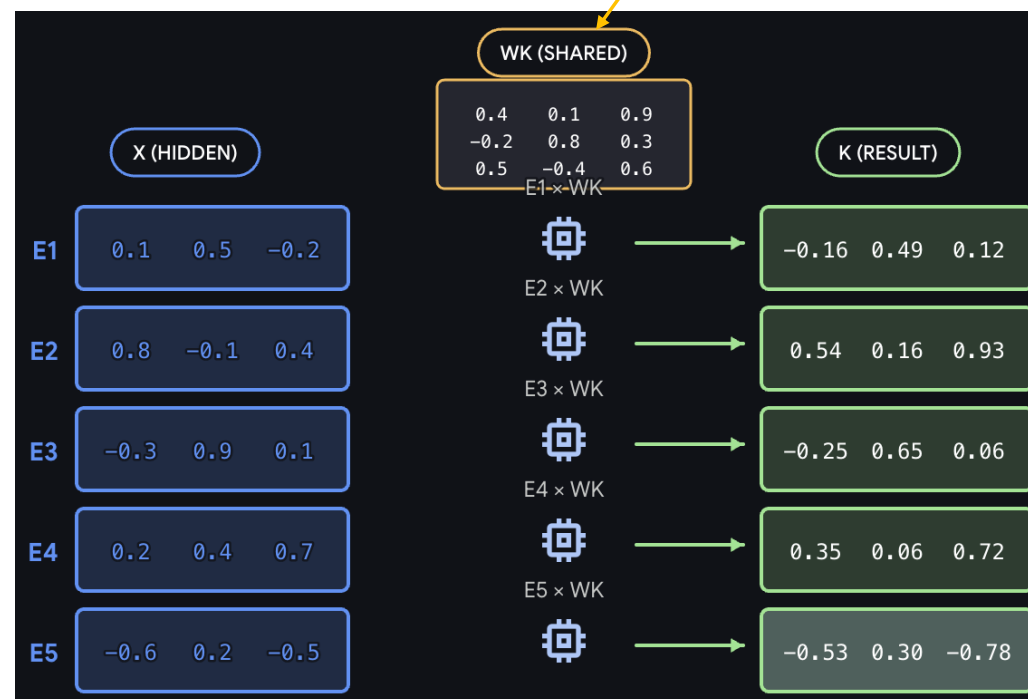
- In the prefill phase, the calculations for all input tokens can be executed in parallel (across all prompt tokens and all heads, but not over all layers).

- High GPU utilization.
- Compute-bound phase.
- Determines TTFT (Time to First Token).

What is the biggest animal?

E1 E2 E3 E4 E5

for one head at one layer



WK

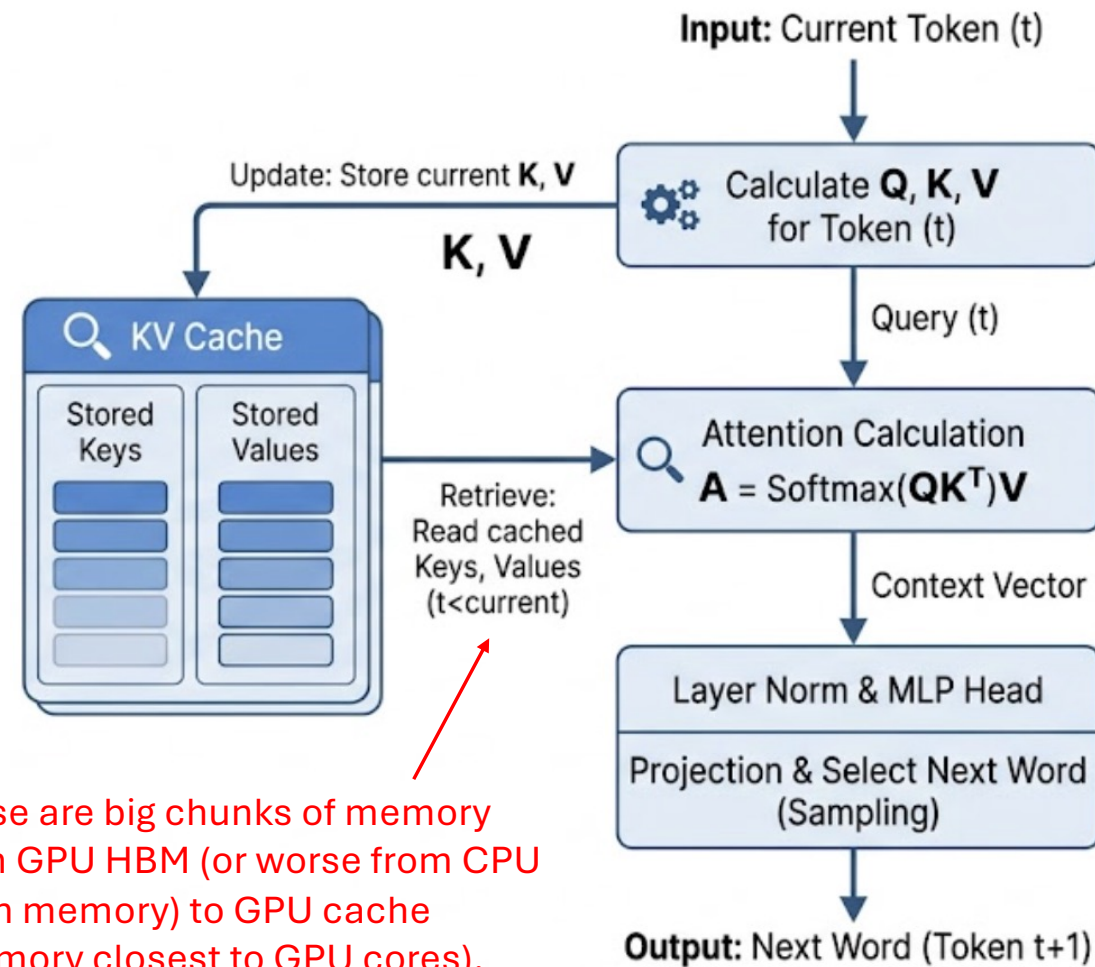
Parameters for generating keys.

K

KEY

Phase 2: Decode (Generation)

- Computing later output token
- Low GPU compute utilization.
 - Each decode step processes only 1 token.
- Memory-bandwidth bound.
 - Every step must load the entire KV Cache from High-bandwidth memory (HBM, e.g., A100 has 80GB HBM); the GPU spends most of its time waiting for memory transfer, not computing.
- Determines (Time Per Output Token).
 - (TPOT > 200ms feels sluggish to users; production systems like GPT-4o achieve ~30–50ms (~20 tokens/sec))

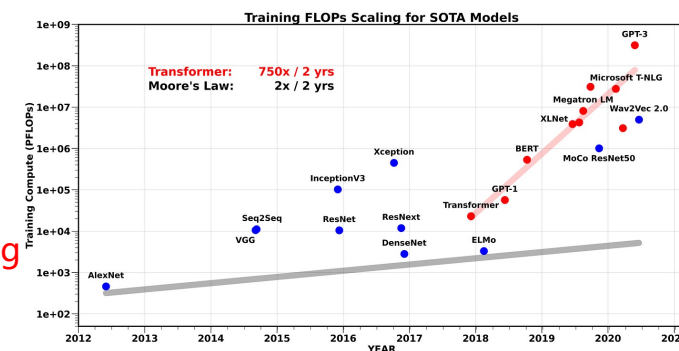


The "Memory Wall" in LLMs

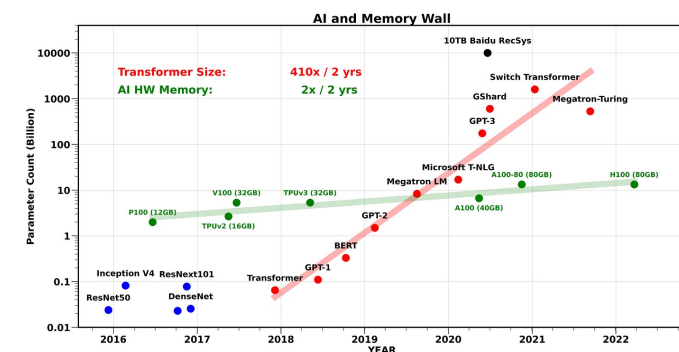
- **Massive Data Load:** Generating a new token forces the historical KV cache (of size $L * d$) to be re-read from memory, a multi-gigabyte operation with a moderate large context length.
 - L : Context/sequence Length, d : dim
- **1 GB Loaded per Step:** A real example for Llama-3 8B at seq=4096.
- The gap between GPU core compute and memory transfer speed growth.
- **Compute Starvation:** the compute power sits idle while waiting for the slow memory bandwidth to fetch the cache.

<https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8>

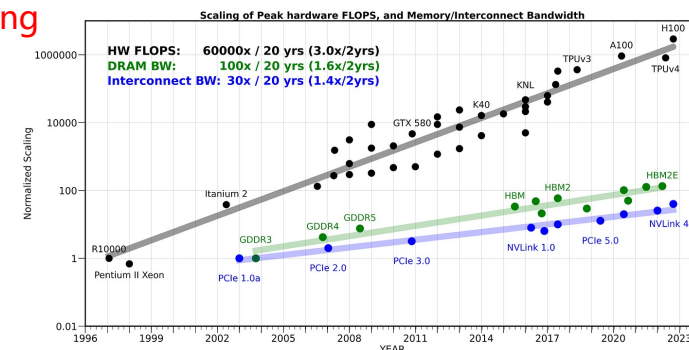
FLOPs needs grows faster than computing power



Memory needs grows faster than GPU memory

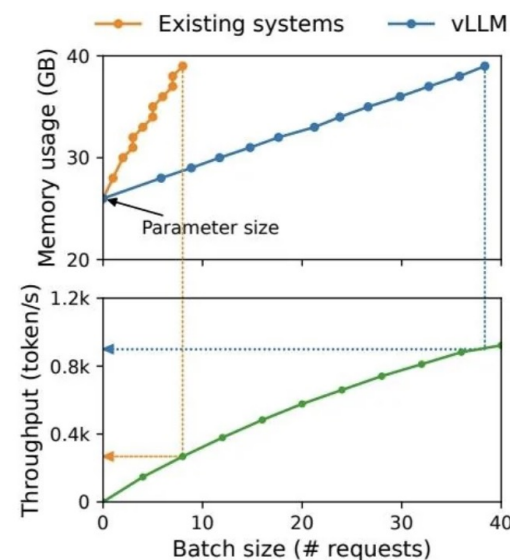
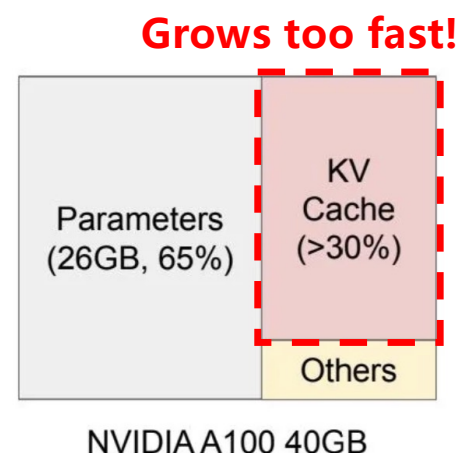


Computing speed grows faster than memory transfer speed



Linear Growth, Massive Impact

- As sequence length L increases, the KV Cache grows linearly, acting as a hard limit on the GPU's context window.
- Weights are fixed, but long document tasks crash with CUDA out of memory. The culprit is the KV Cache, not model size.
- Memory layout when serving an LLM with 13B parameters on NVIDIA A100.



vLLM stands for Virtual Large Language Model and supports LLMs in inferencing and model serving efficiently.

Kwon, Woosuk, et al. "Efficient memory management for large language model serving with pagedattention." symposium on operating systems principles. 2023.

How big is the KV Cache

- The size formula (FP16)

$$\text{Size} = \text{Batch} \times \text{SeqLen} \times 2 \times \text{Layers} \times \text{Heads} \times \text{Dim} \times 2 \text{ bytes}$$

Batch: The number of concurrent sequences being processed.

SeqLen: The max number of tokens of the sequences.

2: the two separate matrices need to cache: K (Key) and V (Value).

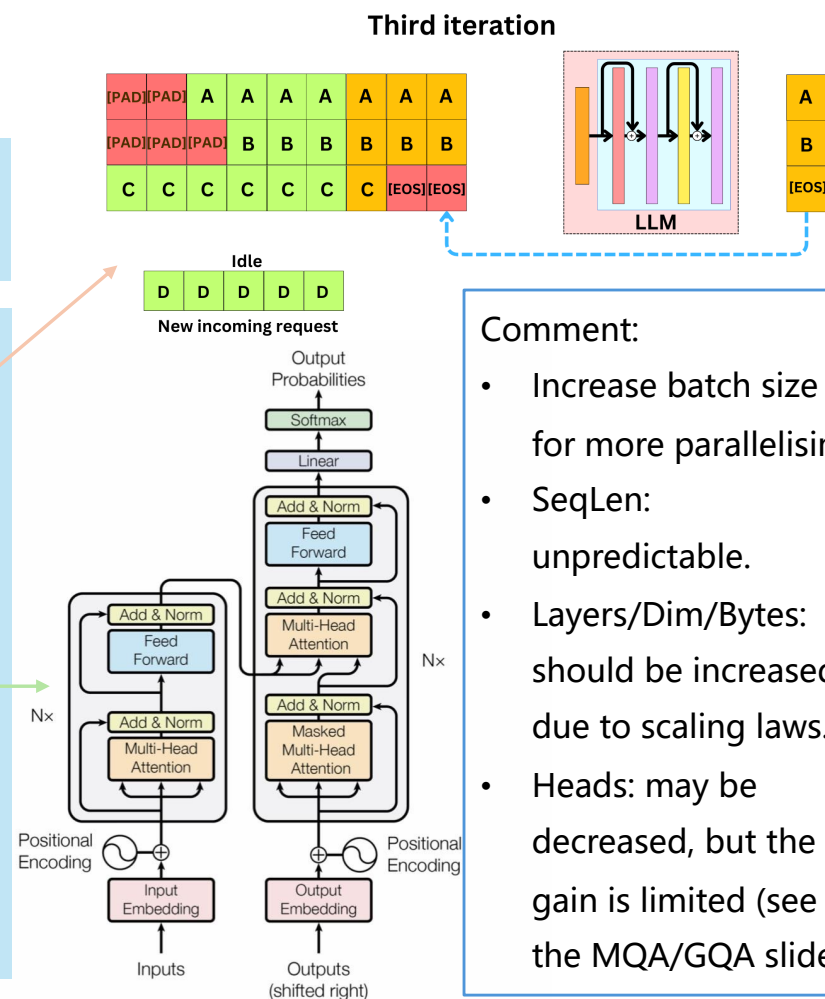
Layers: The number of Transformer blocks.

Heads: The number of attention heads in multi-head attention.

Dim: The hidden dimension size of each attention head.

2 bytes (Data Precision): for storing a FP16 number.

A batch of 3 sequences. LLM generates a token for each sequence in parallel in iterations.

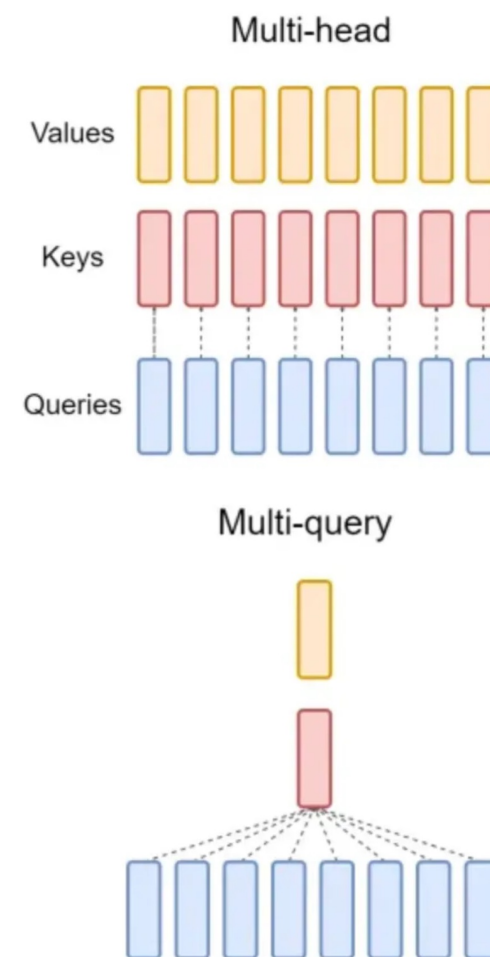


FlashAttention

- Tiling: compute attention softmax online block-by-block and don't cache the attention weights at all.
 - A simple demonstration
 - Suppose we have 3 keys/values, and a query generates 3 logits $[m_1 \ m_2 \ m_3]$.
 - Softmax calculates the weights over 3 value vectors.
 - $[w_1 \ w_2 \ w_3] = (\frac{\exp(m_1 - m^*)}{d}, \frac{\exp(m_2 - m^*)}{d}, \frac{\exp(m_3 - m^*)}{d})$
 - $d = \sum_{i=1,2,3} \exp(m_i - m^*)$
 - $m^* = \max(m_1 \ m_2 \ m_3)$
 - Want to find the weighted sum $s = w_1 v_1 + w_2 v_2 + w_3 v_3$
- The nominators can be calculated in parallel, but the denominator d is the sum of all nominators: the weights can be found only after going through all logits.
- What if the logits are in different places?
- Define the partial sum $d_j = \sum_{i=1}^j \exp(m_i - m^*)$
- $s_j = ((s_{j-1} * d_{j-1}) + \exp(m_j - m^*) v_j) / d_j$

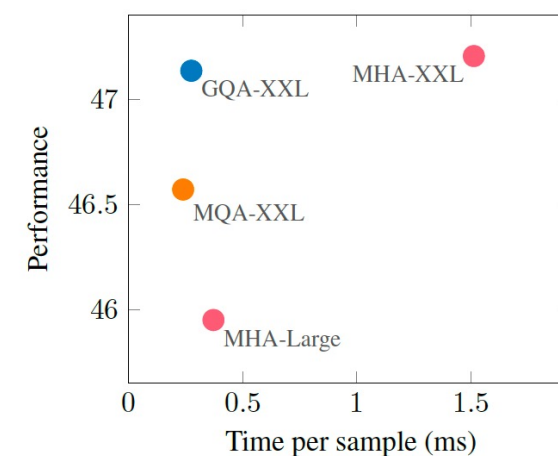
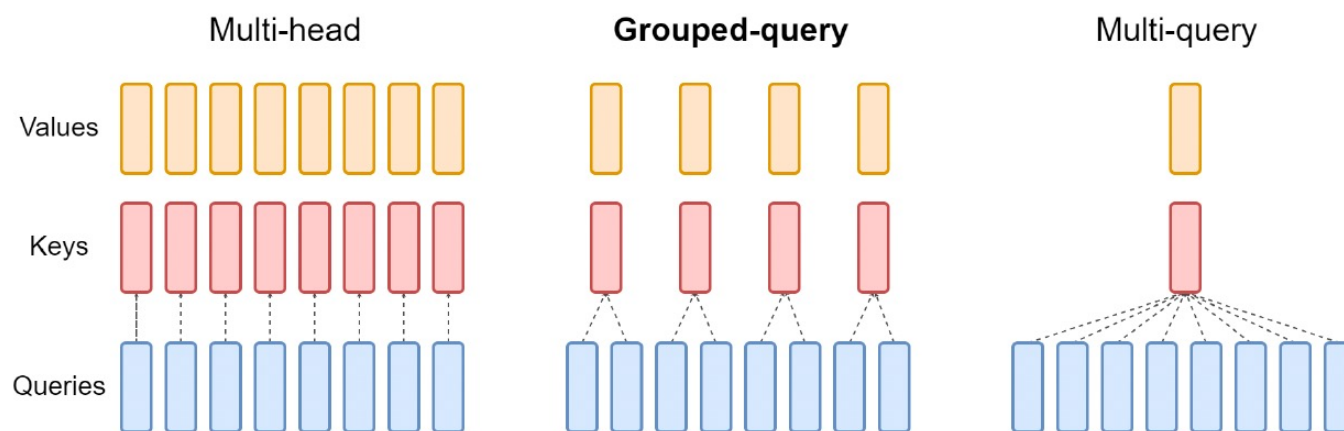
Optimizing Attention Heads

- Multi-Head Attention (MHA): Each Query has its own dedicated Key and Value.
 - The KV Cache expands drastically, serving as the primary culprit behind the "memory wall" we encounter.
- To reduce the KV-cache bottleneck in MHA, **Multi-Query Attention (MQA)** where the keys and values are shared across all of the different attention heads.
 - KV Cache Compressed to the Extreme.
 - Can impact the model's expressive power and generation quality.



Optimizing Attention Heads

- Grouped Query Attention (GQA)
 - An interpolation between MHA and MQA by introducing subgroups of query heads. Each group has a single Key and Value head.
 - In contrast to MQA, GQA keeps the same proportional decrease in memory bandwidth and capacity as model size increases.

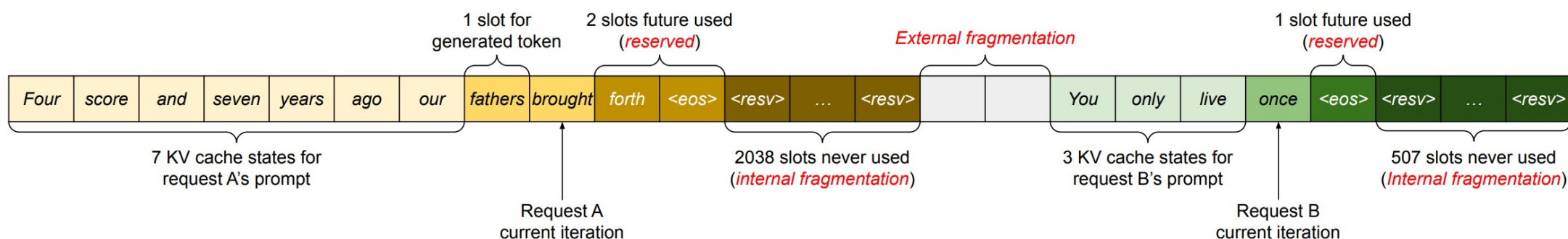


Spend much less time but with slight performance reduction.

Image source: <https://medium.com/@atulit23/implementing-multi-head-latent-attention-from-scratch-in-python-1e14d03fbc91>

The Memory Waste: Why Fragmentation Happens

- An LLM may allow a max length of 2048 tokens.
- But in reality: User A sends 50 tokens. User B sends 300 tokens. User C sends 700 tokens.
- **Internal fragment:** LLM *reserves* 2048 tokens worth of memory for each user. However, users can use much less, resulting in memory fragment due to the *unused reserved* full chunk, and other threads cannot use the fragments. (Output length is unpredictable).
- **External fragment:** operating system (CUDA) may place the memory space of request A and request B at non-consecutive physical space. The gap between them may be too small to be used by another thread and becomes a fragment.

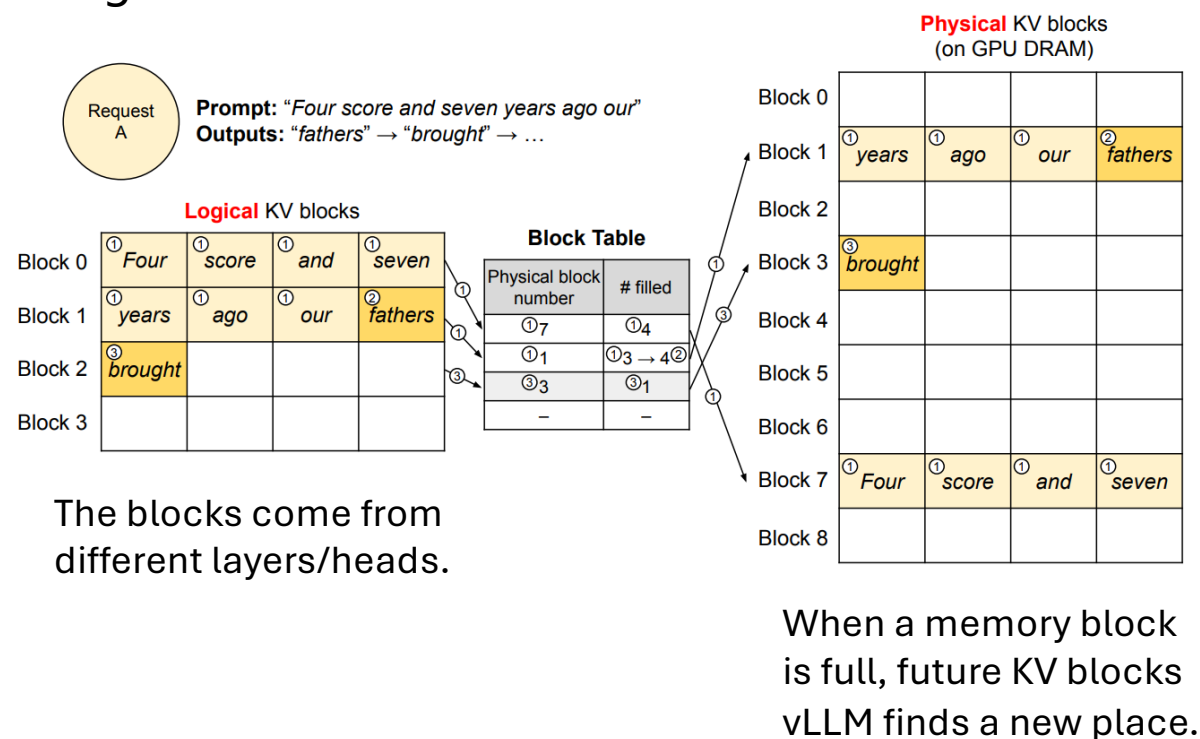


Kwon, Woosuk, et al. "Efficient memory management for large language model serving with pagedattention." symposium on operating systems principles. 2023.

PagedAttention (One request:)

- Prompt: " Four score and seven years ago our"
- A request as a process.

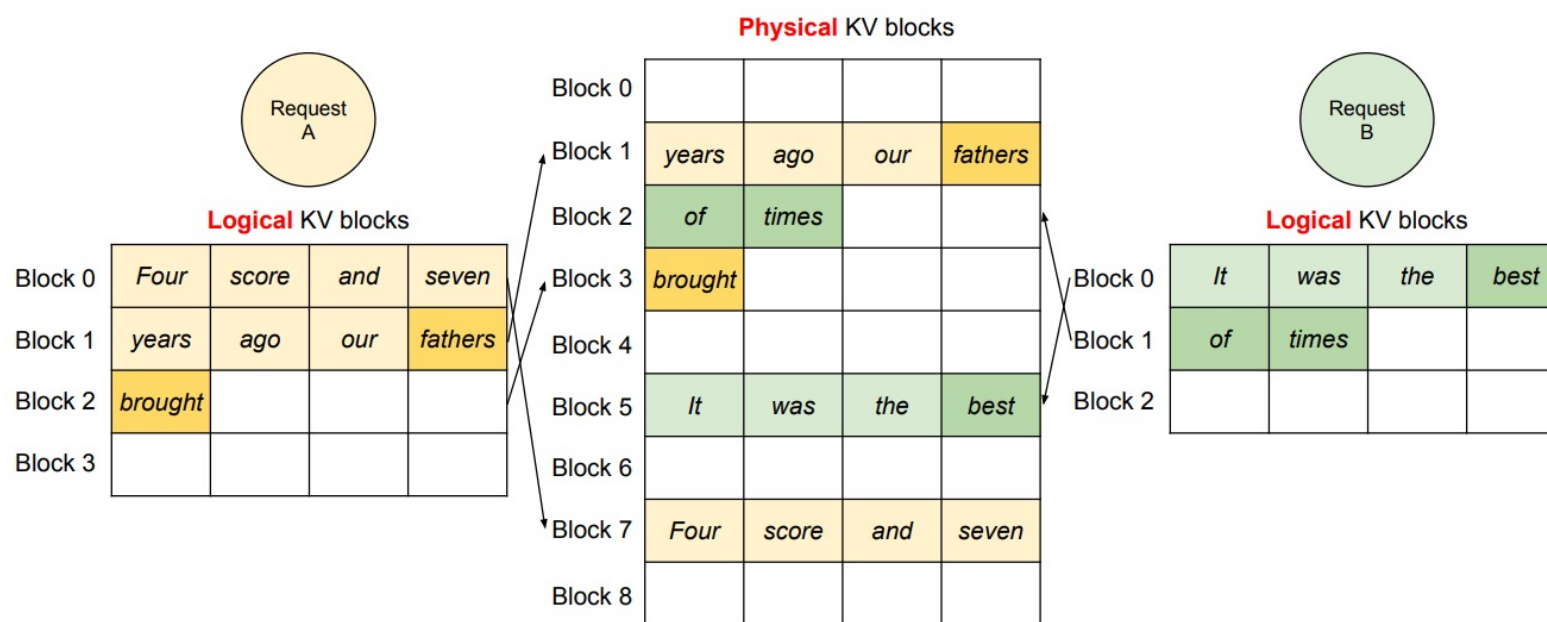
- Logical memory as virtual memory within an operating system.
- Block table translates virtual memory to physical memory.
- Physical memory blocks reside in the GPU's memory, and each block is analogous to a page in virtual memory.



Kwon, Woosuk, et al. "Efficient memory management for large language model serving with pagedattention." symposium on operating systems principles. 2023.

PagedAttention (Multi requests:)

- The logical blocks of the two sequences are mapped to different physical blocks within the space reserved by the block engine.

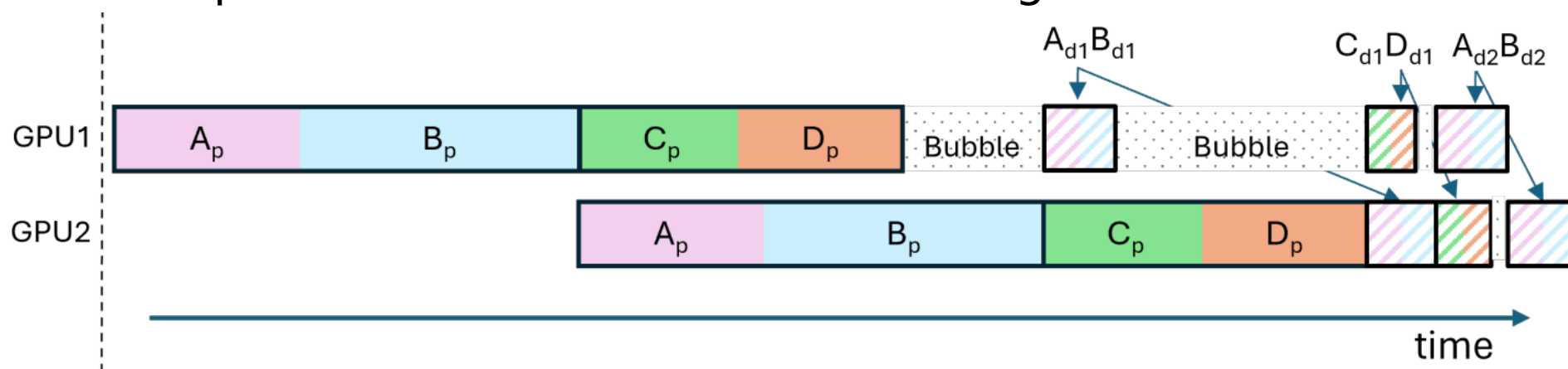


Storing the KV cache of two requests at the same time in vLLM.

Kwon, Woosuk, et al. "Efficient memory management for large language model serving with pagedattention." symposium on operating systems principles. 2023.

Bubbles and Preemption in Continuous Batching

- The decoding iterations of a request (prompt) needs to wait for other's pre-fill to be done: this is a scheduling issue.



Iteration-level Scheduling leads to significant GPU resource underutilization and 'Bubbles'.

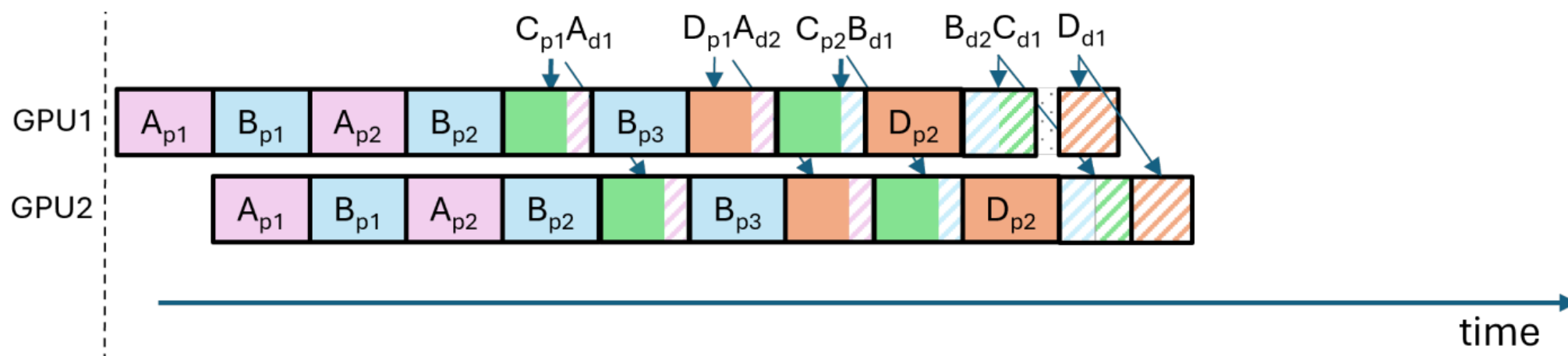
Requests: A, B, C, D, where for a request R, R_p is the prefill portion (can be of varying size), and R_{d1}, R_{d2}, ... are the subsequent decode tokens. The prefill portion of a request, R_p, can be split into smaller chunks R_{p1}, R_{p2}, ..., as shown below.



Agrawal, Amey, et al. "Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills." arXiv:2308.16369 (2023).

Chunked prefills with decode-maximal batching

- Chunked Prefills: Instead of processing the entire Prefill task at once, it is split into fixed-size chunks.
- Piggybacking: When processing a specific Prefill chunk (C_{p1}), utilize its remaining computational capacity to "piggyback" an ongoing Decode task (A_{d1}).



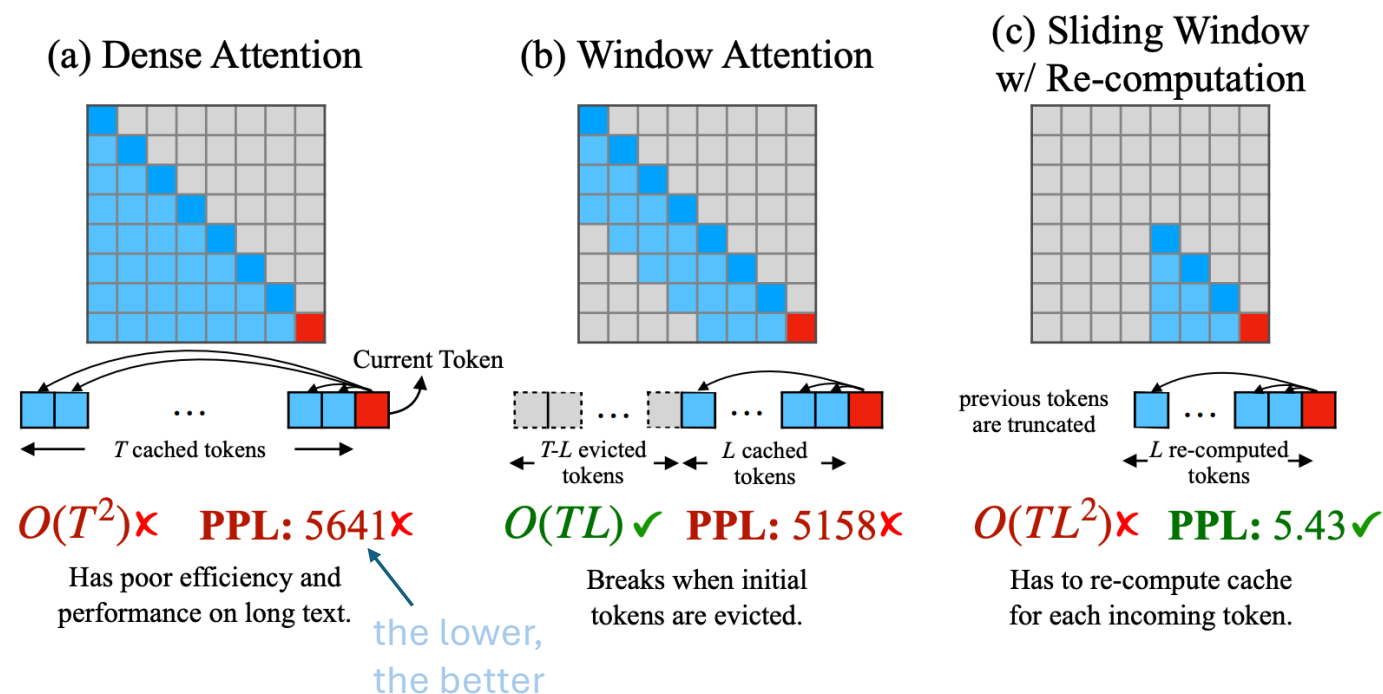
significantly reduces pipeline bubbles and enables more efficient piggybacked decodes.



Agrawal, Amey, et al. "Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills." arXiv:2308.16369 (2023).

Memory Wall in Long Context

- When the text length exceeds the GPU memory capacity, the system will immediately encounter an Out of Memory (OOM) error
- Limit the window size:



(a): Memory explodes; fails beyond pre-train length.

(b): Collapses immediately when first tokens are evicted.

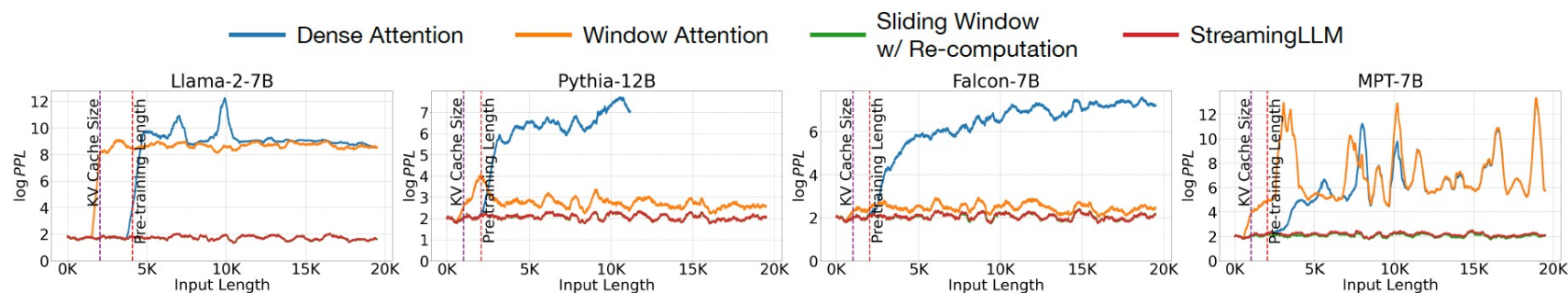
(c): Prohibitively slow due to repeated $O(L^2)$ recalculation.

PPL: Perplexity, the lower the better.

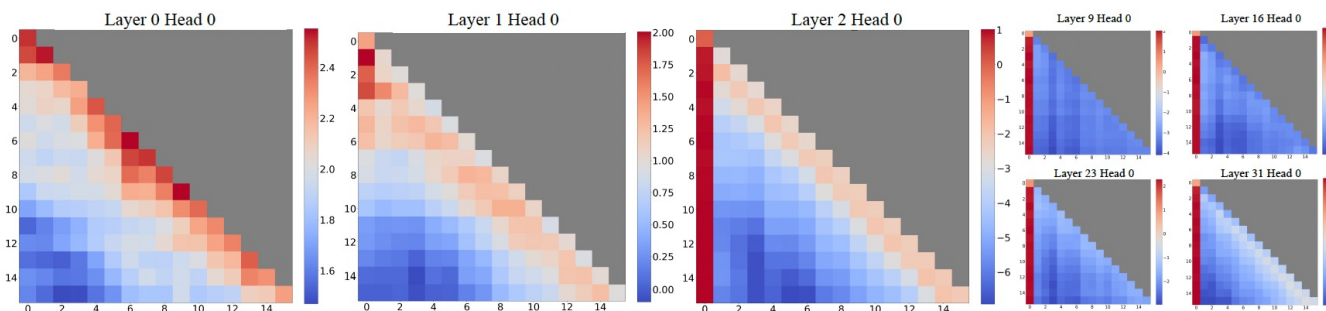
Xiao, Guangxuan, et al. "Efficient Streaming Language Models with Attention Sinks." ICLR 2024

StreamingLLM: Attention Sink is All You Need

With a window of limited size, the model performance drops due to missed information.



One interesting observation: attention sink.



Hypothesis: due to the normalization of softmax, though the initial tokens do not carry much semantics information, the position embedding instructs the attention weights to be placed at the beginning.

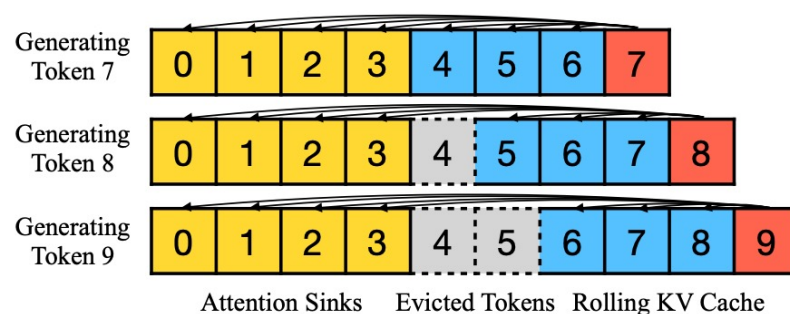
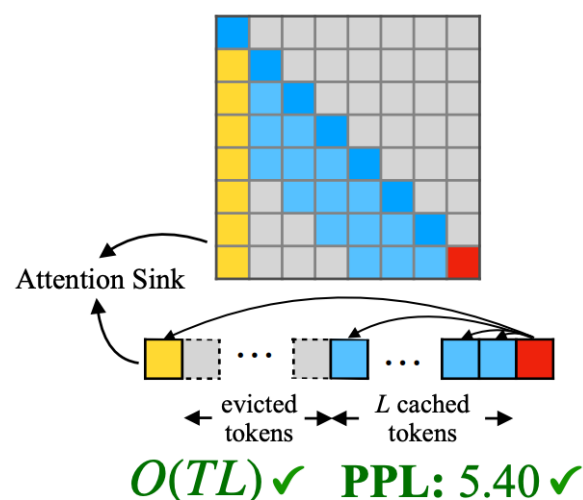
Why: as during auto-regression training, the initial few positions are seen the most often.

Xiao, Guangxuan, et al. "Efficient Streaming Language Models with Attention Sinks." ICLR 2024

StreamingLLM: Attention Sink is All You Need

- Key insight: The initial tokens are visible to all subsequent tokens; consequently, the model tends to focus its attention on these initial tokens, forming an "attention sink."

(d) StreamingLLM (ours)



The KV cache of Streaming LLM (now of a fixed size).

Llama-2-13B	PPL (↓)
0 + 1024 (Window)	5158.07
4 + 1020	5.40
4"\n"+1020	5.60

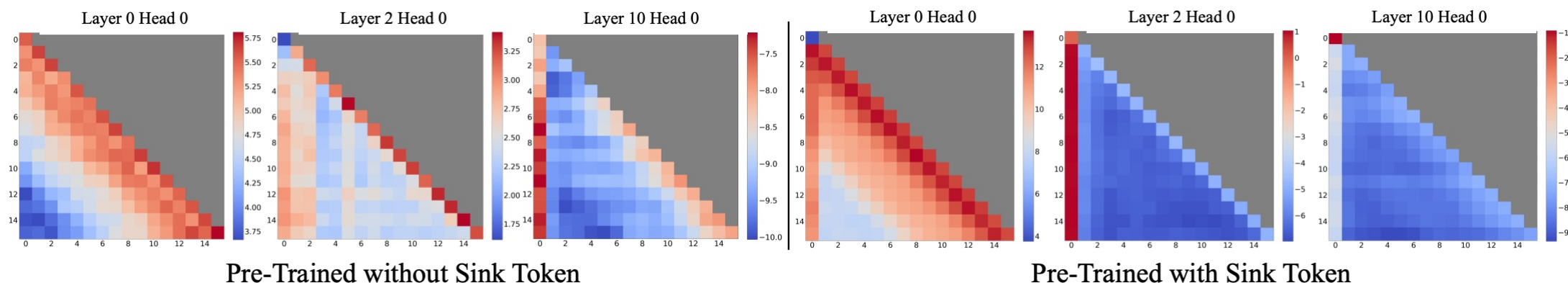
Some dummy characters (e.g., '\n') also maintain the performance (PPL)

Keeps the attention sink (several initial tokens) for stable attention computation, combined with the recent tokens.

Xiao, Guangxuan, et al. "Efficient Streaming Language Models with Attention Sinks." ICLR 2024

Attention Map

- Visualization of average attention logits over 256 sentences, each 16 tokens long, comparing models pre-trained without (left) and with (right) a sink token.



- Without a sink token, models show local attention in lower layers and increased attention to initial tokens in deeper layers.
- With a sink token, there is clear attention directed at it across all layers, effectively collecting redundant attention.
- With the presence of the sink token, less attention is given to other initial tokens, supporting the benefit of designating the sink token to enhance the streaming performance.

Xiao, Guangxuan, et al. "Efficient Streaming Language Models with Attention Sinks." ICLR 2024

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (short answer): write down the attention equation involving Q, K, V matrix $\text{Attention}(Q, K, V)$.
 - Q2 (T/F): exact online computation of softmax is impossible and approximation of softmax is needed when calculating the normalization factor online.
 - Q3 (multi-choice): which of the following is a factor about the size of KV cache: (A) number of layers; (B) layer normalization parameters; (C) the differences in the length of user queries; (D) the length of the generated token sequences; (E) the size of GPU memory.
 - A: see page 3's first equation; F; (AD)